EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

The second half is office hours

First	Aider as the worked example: what the model sees, how a reply becomes a file change, how to read a failure
Then	The hackathon 1 briefing, and what you own for the build
Rest	Office hours, in this room. Your work, your questions, staff circulating

Nothing is collected at the end of this lab.

THE RUNNING EXAMPLE

One request touches three files

```
Add a --priority flag to taskr's add command.
```

taskr/cli.py
Accepts the flag and its default.

taskr/task.py
Carries the value on a task.

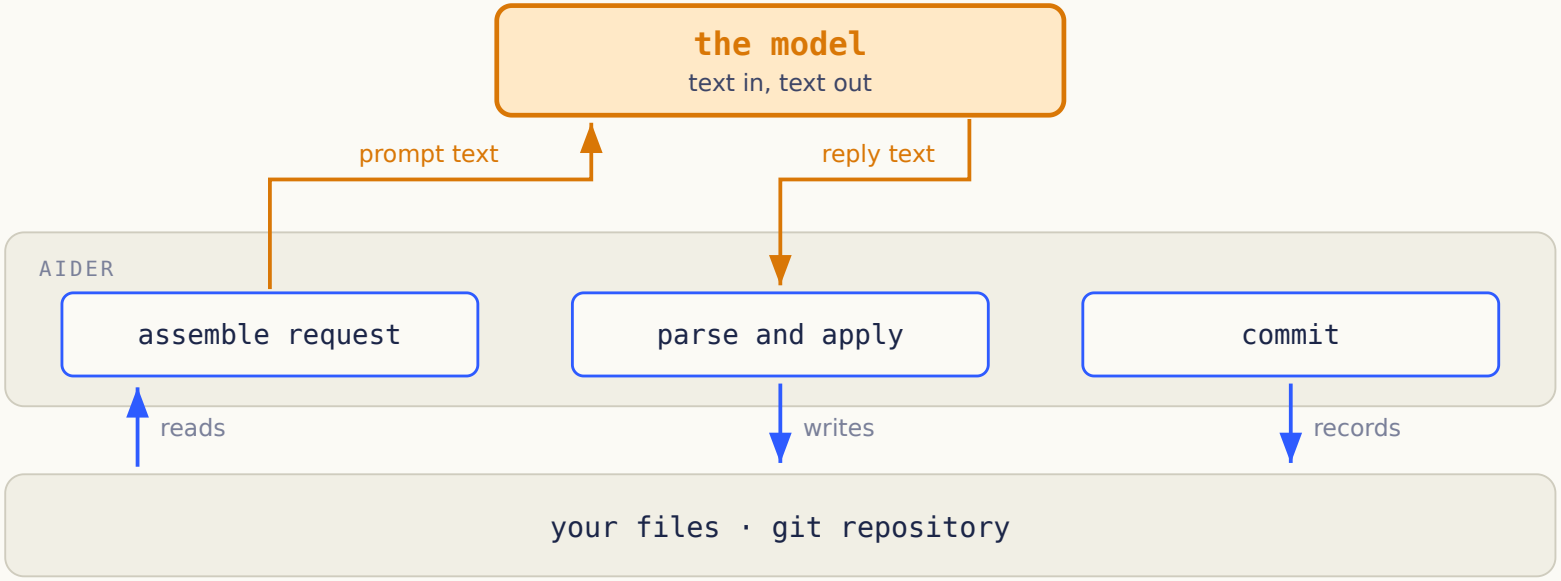
taskr/store.py
Writes it down and reads it back.

One turn

SELECTED FILES TO COMMIT

WHO DOES WHAT

Only Aider touches your files



No line runs from the model to your repository. You pick the task and judge the result.

STEP 1 · ASSEMBLE

Aider builds the request from parts you chose

- The instructions that define the edit format
- The files you added, in full
- A repository map, when it is enabled
- The chat history from this session
- Your new message

Context is a selection. A file on disk that you never added is not in it.

STEP 1 · SELECTION

The /add command decides what the model reads

```
/add taskr/cli.py taskr/task.py taskr/store.py
/read-only specs/priority.md
/tokens
```

Added files can be rewritten. Read-only files can only be cited.

Naming a path in a sentence does not load it. Only these commands do.

The repo map carries signatures without bodies

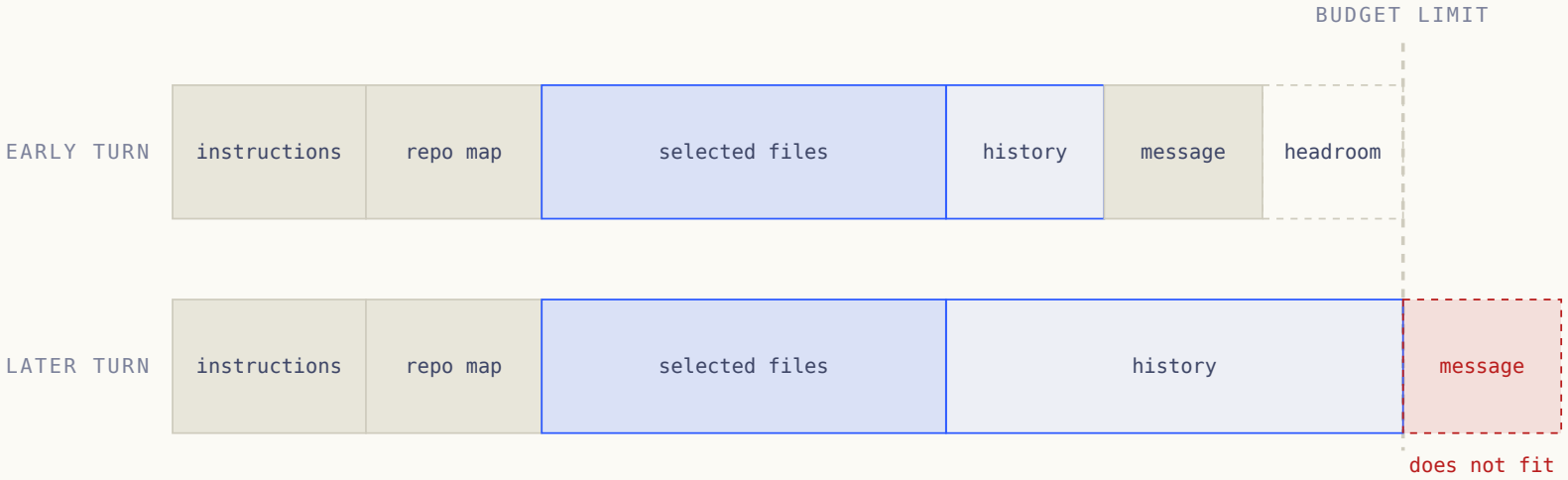
```
taskr/store.py
  Store.add(title, tags) -> Task

taskr/task.py
  Task: id, title, tags
```

Aider builds a graph of definitions and references, then ranks the useful parts into a token budget.

A signature tells you where to look. It does not tell the model how the function works.

STEP 1 · BUDGET

Everything you add spends the same budget



History and the repo map spend the same budget as the file you actually care about.

The model answers with edit-shaped text

```
taskr/cli.py
<<<<<<< SEARCH
add.add_argument("title")
=======
add.add_argument("title")
add.add_argument("--priority", default="normal")
>>>>>>> REPLACE
```

This block adds a flag to the CLI. Nothing stores the value and nothing validates it.

STEP 3 · PARSE

Aider parses the reply before it trusts it

Parsing asks: is this a block?

A missing divider is a syntax error. Report it and keep running.

Validation asks: does it apply?

An unknown path, or zero exact matches, fails later and for a different reason.

Two questions, two error messages. A student who merges them writes one useless message for both.

STEP 4 · APPLY

An edit lands only on an exact match

On disk

```
add.add_argument("title")
```

In the SEARCH block

```
add.add_argument( "title" )
```

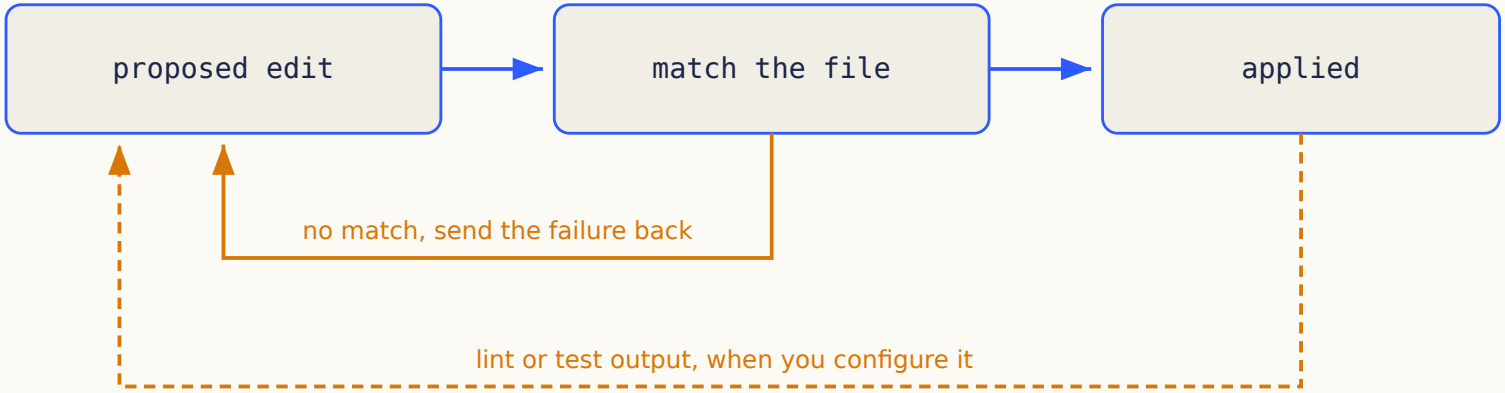
Two extra spaces. Zero matches. The block is well formed and cannot be applied.

Aider can fall back to looser matching, and it can keep the blocks that did apply.

Your Stage 1 contract does neither. Exact matches only, and all blocks or none.

STEP 5 · REPAIR

A failed edit becomes the next prompt



Aider chooses the next action here from a fixed set. It is a repair cycle, not a tool the model picked.

STEP 6 · COMMIT

Every accepted edit becomes one commit

```
* 8f2a1c3  aider: add --priority to the add command
* 41b9e07  aider: store priority on Task
* 0c7d5e1  initial taskr
```

Undo means resetting to the parent of a commit your own session created.

Git reverses file changes. It does not reverse an email that has already been sent.

CHECK · 3 MINUTES

The six steps account for the whole feature

For `--priority`, answer with the person next to you:

1. What entered the context, and what did not?
2. Which program read the reply?
3. What could look correct and still be wrong?
4. What evidence would show the feature works?

Diagnosis

READ THE FAILURE BEFORE YOU RE-PROMPT

DIAGNOSIS

Each failure has its own symptom

What you see	What to check first
It calls a function that does not exist	The file defining it was never added
The block looks right and will not apply	The file changed after the model saw it
Tests pass, the feature is still broken	The tests never covered the missing part
The request times out	The endpoint URL, the served model, the input size

More than one cause can produce the same symptom. Name the evidence that separates them.

DIAGNOSIS

Aider improves results without a better model

A chat window

You paste code in. It answers with code. You copy the answer back and hope you pasted the right version.

Aider, same model

It reads the files you selected, answers in a fixed edit format, applies the result and commits it.

The model still has limits. You can measure them once the harness stops being the variable.

PRACTICE · 6 MINUTES

The priority value disappears on restart

```
$ taskr add "ship lab" --priority high
added #4
$ taskr list
#4  ship lab   priority: high

$ taskr list        # new process, same database
#4  ship lab   priority: normal
```

What do you inspect first? Write the acceptance criterion that was missing, and the test that would have caught it.

SEPTEMBER 24

Hackathon 1 runs in the browser workspace

- Two hours, during the session, on your own project
- The feature prompt is revealed in the room
- You submit before you leave
- Graded on its own, separately from the build

Bring a runnable increment. There is no separate hackathon project to prepare.

The build

DESIGN FIRST, THEN IMPLEMENT IN INCREMENTS

The packet gives you specs and no code

In the packet

SPEC.md, RUBRIC.md, ACCEPTANCE.md, EXAMPLES.md, DESIGN-GUIDE.md, WORKFLOW.md, and the Aider configuration you build with.

Not in the packet

Application code. Tests. A module layout. A conversation loop. A diagram of the answer.

You run `aider` to build it. You write `bin/assistant` to run what you built.

Your assistant reads files and proposes edits

```
/files                → list selected files
/add greet.py         → include this file in model requests
What does greet do?   → stream an answer about the selected code
Change Hello to Hi.   → validate the reply and preview a diff
```

A terminal assistant over selected text files. Not all of Aider.

THE SAFETY RULE

Nothing changes on disk until you approve it

Point in the interaction	`greet.py`	Git
Diff shown, waiting	Still `Hello`	No new commit
You approve	Now `Hi`	One owned edit commit
You decline	Still `Hello`	No new commit
You undo the approved edit	Back to `Hello`	Reset to the parent commit

One invalid block cancels the whole proposal, including the blocks that were fine.

SCOPE

Seven features define the build

Feature	Required behavior
F1	Conversation, budget handling, streamed replies
F2	Add, drop, list and clear context
F3	Current file contents, rendered consistently
F4	An edit-format prompt you have tested
F5	Parsing with useful errors
F6	Exact edits, complete preview, explicit approval
F7	One commit per accepted edit set, guarded undo

CONFIGURATION

One YAML file selects the endpoint and model

```yaml
base_url: https://api.example.edu/v1
model: course-model-id
api_key_env: ASSISTANT_API_KEY
context_budget: 8000
timeout_seconds: 60
temperature: 0.2
```

The schema is fixed. How you load, represent and validate it is yours.

OWNERSHIP

You own the architecture

We specify	You design
Observable behavior and safety rules	Responsibilities and who owns state
The API and configuration contract	Internal interfaces and error flow
Acceptance scenarios and the rubric	Tests, fixtures, increment boundaries
Where the course is going	Where later capabilities enter

Different architectures can earn full marks. Designing for a future you cannot describe cannot.

SPECIFICATIONS

Specs come in two sizes

System spec, written once

Behavior, architecture, interfaces, failure handling, verification, and why you chose it.

Increment spec, one per change

Scoped context, ordered tasks, and a result you can check when it is done.

Commit the design before the application code. Revise it when evidence changes a decision.

Three diagrams answer three questions

Diagram	The question it answers
Components and data flow	Who owns state, and what crosses each boundary?
Request sequence	Who does what during one proposed edit?
Edit lifecycle	When can an edit be approved, rejected or undone?

Mermaid is enough. Keep the first versions in git and update the final ones to match what you built.

BUILD RUBRIC

Specification and diagrams carry 40 points

Criterion	Points
Requirements and acceptance criteria	10
Architecture and interface design	10
Aider implementation specs	10
Diagrams: components 4, sequence 3, lifecycle 3	10

Completeness and usefulness are graded. Document length is not.

BUILD RUBRIC

Behavior and verification carry 40 points

Criterion	Points
Required functionality, scored feature by feature	20
Behavioral tests	8
Safety and recovery tests	7
Fake integration 3, live evidence 2	5

A correct failing test can earn test credit while exposing an unfinished feature.

BUILD RUBRIC

Evidence and reconciliation carry 20 points

Criterion	Points
Spec-first history	5
Deliberate Aider use	5
Diagnosis and design reconciliation	5
Operating instructions	3
Model disclosure and evidence index	2

Evidence means a pointer someone can follow. Volume of logs is not evidence.

The model rule covers your specification too

- Drafting, critique, diagrams, code and tests all fall under it
- The 9B is recommended, the 4B is permitted
- Secondary models get recorded the same way
- Any compatible hosting service is allowed

A different provider does not authorize a different model.

THROUGH DECEMBER

One repository carries you to December

Start with one small result you can check. Refactor when you need to, and keep the phase snapshots.

What comes next

THE SAME REQUEST, IN A DIFFERENT KIND OF TOOL

Claude Code lets the model pick the next step

Aider	A general agentic CLI
You frame a bounded edit task	You hand over a broader task
Selected files and a repo map	Tools that discover and read files
A fixed repair cycle	The model picks the next tool
You check between tasks	Permissions and stop rules bound it

An agentic CLI runs the whole request itself

```
grep -rn "add_argument" taskr/   → finds cli.py
read taskr/store.py              → finds the save path
edit cli.py, task.py, store.py
pytest -q                        → 1 failed
edit store.py
pytest -q                        → passed
```

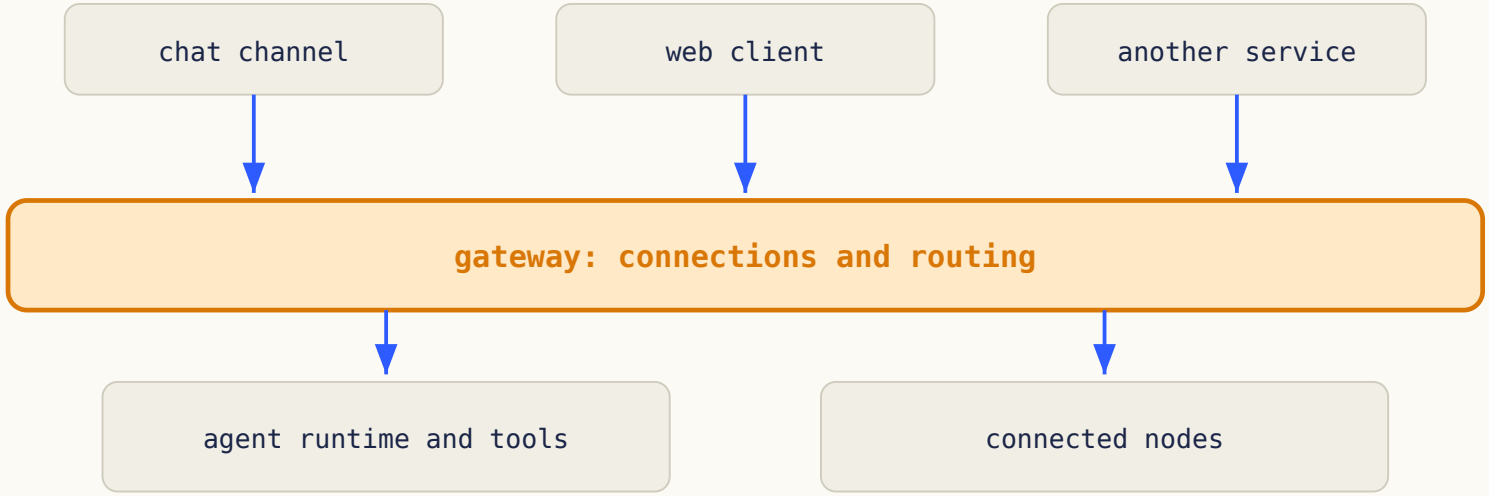
Each result is evidence for the next decision. Who stops a bad one?

Written instructions still need a harness

- Project instructions give persistent guidance
- Skills package instructions for a particular task
- Hooks attach checks to events
- Tool integrations expose actions and results

Prose asks. Code and permissions enforce.

A gateway gives one agent many ways in



DESIGN FOR IT NOW

Your design names where new capability enters

- Where could a model-selected tool join your control flow?
- Could you swap the endpoint client without touching state management?
- Could a web interface reuse the same application behavior?

Answer with a boundary, not an implementation. You are allowed to change your answer in November.

Office hours

THE REST OF THE SESSION IS YOURS

OFFICE HOURS

Spend this time on whatever is in your way

Staff are in the room. Good things to use them for:

1. Your repository access or tooling not running
2. A requirement in the packet you read two ways
3. The boundary of your first increment
4. A design decision you want argued with

Nothing here is collected. Start wherever your build is actually stuck.

WORTH DOING

A partner finds what you stopped seeing

If you want a second pair of eyes before you leave, trade these three:

1. What has to be true when this increment is done?
2. Which decision would the model still have to guess?
3. What test would catch a plausible mistake here?

Optional, and not recorded anywhere. Bring anything ambiguous in the packet to staff instead.

THE BUILD · DUE FRI SEP 25

Finish the design before you write code

1. Complete the system design and the three diagrams
2. Review them in a fresh permitted-model session
3. Commit the design and the first increment spec

Then build, test and revise one increment at a time.

This is the build talking, not this lab. Submission packet, deadlines and exact scoring all live in your repository.

Questions?